Activity Management

1. Document Purpose

This document defines the requirements for the **Activity Management** feature in the iOS and Android CRM applications, enabling users to log and manage customer interactions directly from mobile devices.

---

2. Feature Overview

Activity Management allows users to manually log and manage customer-facing interactions such as calls, meetings, emails, and follow-ups against CRM records. This feature ensures sales and support teams can record interactions in real time while on the move.

---

3. Goals and Objectives

Goals

- Enable real-time activity logging from mobile.
- Maintain parity with existing web CRM activity functionality.
- Reduce missed or delayed activity entries.

Success Metrics

- Increase in activities logged from mobile.
- Reduction in post-interaction logging delays.
- User adoption of mobile activity logging.

---

4. In-Scope Functionality

4.1 Supported Activity Types

- Call
- Meeting
- Email

- Follow-up task
- Manual note

4.2 Activity Creation

Users must be able to:

- Create activities from within a CRM record.
- Associate each activity with a single CRM record.
- Capture required fields:
    - Activity type
    - Date and time
    - Duration (where applicable)
    - Notes or description
    - Activity owner

4.3 Activity Editing and Deletion

- Users can edit or delete activities they own, subject to permissions.
- Activity updates must reflect in real time on web and mobile.

---

5. User Flows

Log Activity

1. Open CRM record.
2. Select “Add Activity”.
3. Choose activity type.
4. Enter details.
5. Save activity.

---

6. Functional Requirements

- Support offline activity creation.
- Sync offline activities automatically when online.
- Prevent duplicate activities during retries.

- Validate required fields before saving.

---

7. Non-Functional Requirements

Performance

- Activity creation must complete within 1 second online.
- Offline saves must be instantaneous.

Security

- Enforce role-based access control.
- Activity ownership must be preserved.

---

8. Out of Scope

- Automated activity creation.
- Activity analytics and reporting.
- Email auto-sync from inbox.

---

9. Acceptance Criteria

- Users can log and edit activities from mobile.
- Activities sync correctly across platforms.
- Permissions are respected.

---

10. Dependencies

- Activity APIs from web CRM.
- Offline sync framework.
- Authentication and authorization services.

API Contracts Activity Management

1. Activity APIs

Create Activity

POST /api/v1/activities

Request

```
{
  "record_id": "string",
  "record_type": "LEAD | CONTACT | ACCOUNT | OPPORTUNITY",
  "activity_type": "CALL | MEETING | EMAIL | FOLLOW_UP | NOTE",
  "start_time": "ISO-8601",
  "duration_minutes": 15,
  "description": "string",
  "client_request_id": "uuid"
}
```

Response

```
{
  "activity_id": "string",
  "status": "CREATED",
  "created_at": "ISO-8601",
  "updated_at": "ISO-8601"
}
```

Update Activity

PUT /api/v1/activities/{activity_id}

Request

```
{
  "description": "string",
```

"duration_minutes": 20

}

Delete Activity

DELETE /api/v1/activities/{activity_id}

Developer Design Document

Activity Management (Mobile CRM)

1. Overview

This document describes the technical design for the **Activity Management** feature in the mobile CRM application. The feature enables users to create, edit, delete, and sync activities associated with CRM records.

---

2. Architecture Overview

Components

- Mobile App (iOS and Android)
- Backend Activity Service
- Offline Storage and Sync Engine
- Authentication and Authorization Service

High-Level Flow

1. User creates or edits an activity on mobile.
2. Activity is saved locally (offline-first).
3. Sync engine pushes changes to backend when online.
4. Backend validates permissions and persists data.
5. Updates propagate to web and other devices.

---

3. Data Flow

Activity Creation

- Mobile UI validates mandatory fields.
- Activity stored in local DB with sync_status = PENDING.
- Sync worker posts activity to backend.
- Backend returns activity_id and timestamps.
- Local record updated to SYNCED.

Activity Editing

- Only allowed for user-owned activities.
- Offline edits create a new sync version.
- Conflict resolution uses last-write-wins.

---

4. Offline and Sync Strategy

- Local storage: SQLite or Room equivalent.
- Sync states:
  - PENDING
  - SYNCED
  - FAILED
- Retry with exponential backoff.
- Idempotency keys used for safe retries.

---

5. Permissions

- Activity visibility governed by record-level access.
- Edit and delete allowed only if:
  - User is owner OR
  - User has edit permission on activity entity.

---

6. Error Handling

- Validation errors shown immediately.
- Sync failures logged and retried silently.
- Hard failures surfaced in activity detail view.

---

7. Non-Functional Considerations

- Max activity payload size enforced.
- Sync must not block UI thread.

- Background sync throttled on low battery.